

CS 5180 - Course Project Requirements

1. Project Goal

CPP aims to build a search engine that allows students to retrieve relevant documents from a financial dataset. This project is part of an educational initiative to promote financial literacy, enabling students to better understand and handle real-world financial situations encountered during their professional lives. To achieve this goal, students will design and implement an Information Retrieval (IR) system that processes textual data, builds an index over a document collection, and ranks documents in response to user queries. The system will be evaluated based on its ability to retrieve relevant documents using IR evaluation metrics.



2. Dataset Description

The dataset consists of a collection of 5,000 financial text documents, 25 static queries, and corresponding relevance judgments. These are provided in separate JSON files.

2.1 Documents

- Each document contains a unique identifier and a text field.

- File:

```
documents.json
{
  "doc_id": "...",
  "text": "..."
}
```

2.2 Queries

- Queries represent user information needs expressed in natural language.

- File:

```
queries.json
{
  "query_id": "...",
  "text": "..."
}
```

2.3 Relevance Judgments

- Binary relevance judgments indicate which documents are relevant to each query - documents not listed are considered non-relevant.

- File:

```
qrrels.json
{
  "query_id": "...",
  "doc_id": "...",
  "relevance": 1
}
```

3. Functional Requirements

The system must:

- load the provided dataset.
- preprocess document text.
- build an inverted index over the document collection.
- support query processing for input queries.
- compute relevance scores.
- rank documents based on these scores.

4. Output requirements

- Return the top 25 ranked documents per query.
- Display results in decreasing order of relevance score.
- Provide a pagination mechanism, showing 5 documents per page, including
 - document identifier (`doc_id`).
 - full text.

5. Evaluation

- Static queries
 - ✓ 25 labeled queries.
 - ✓ Compute AP per query and MAP using ALL relevant documents provided.
- Dynamic Labeled Queries (Presentation Day).
 - ✓ 2-3 labeled queries
 - ✓ Queries follow the same format as `queries.json`.
 - ✓ Compute AP per query using ALL relevant documents provided.
- Dynamic Unlabeled Queries (Presentation Day)
 - ✓ 2-3 unlabeled queries.
 - ✓ Evaluation via visual inspection of ranking results.

6. User Interface

- Free design (graphical or console-based).
- Must support:
 - query input.
 - display of ranked results.
 - pagination.